SEA: Segment, Embed, and Align

คำอธิบายทางเทคนิคเชิงลึกของ Pipeline การจัดตำแหน่งคำบรรยาย
ที่ประยุกต์ใช้กับภาษามือไทย --- พร้อมพื้นฐานจากบทความต้นฉบับ
และผลการทดลองจริงจาก Workspace นี้

รายงานทางเทคนิคอัตโนมัติ

Workspace: NECTEC/SEA

26 เมษายน พ.ศ. 2569

บทคัดย่อ

รายงานนี้อธิบาย *Segment, Embed, and Align* (SEA) pipeline อย่างละเอียดในเชิงเทคนิค ตามที่ใช้งานจริงใน Workspace นี้ และประยุกต์กับวิดีโอภาษามือไทย (TSL) ตัวอย่าง รายงานมีการจัดเป็นสามส่วน: **(i)** บทนำของ SEA ตามที่ตีพิมพ์ในบทความต้นฉบับ *``Segment, Embed, and Align: A Universal Recipe for Aligning Subtitles to Signing"* โดย Jiang et al., 2025 (arXiv:2512.08094) ซึ่งครอบคลุมแรงจูงใจ การนิยามงานอย่างเป็นทางการ คำอธิบายอัลกอริทึมเต็มรูปแบบ ชุดข้อมูล benchmark ทั้งสี่ชุด และผลลัพธ์ SOTA ที่ผู้เขียนเผยแพร่; **(ii)** การแยกย่อยทางเทคนิคของ pipeline ทีละขั้นตอน ตามที่รันจริงใน Workspace นี้ --- input, artefact ระหว่างกลาง, command line, พารามิเตอร์, คณิตศาสตร์ภายใน, ตำแหน่งโค้ด, และ output; **(iii)** รายงานผลการทดลองครบถ้วนที่ได้จากวิดีโอบรรยาย TSL จริง (172 คำบรรยาย, 11 นาที 07 วินาที) ครอบคลุมการทดลองเก้าชุด (A, B1, B2, B_MULTI, C_MULTI, C_MULTI_word, D_ASL, D_ASL_gloss, D_ASL_word) รวมถึงตัวเลขผลลัพธ์จริงที่ได้จากการรัน evaluate_all.py ระหว่างการเขียนรายงานนี้

สารบัญ

I พื้นฐาน --- บทความ SEA 3

1 แรงจูงใจ: ทำไมต้องจัดตำแหน่งคำบรรยายภาษามือ? 3

2 นิยามงานอย่างเป็นทางการและ Pipeline สามขั้นตอน 3

2.1 Input และ Output . . . 3

2.2 สามขั้นตอนแบบ modular . . . 3

2.3 ทำไมถึงใช้งานได้เป็น universal recipe . . . 3

3 Model Segmentation อย่างละเอียด 4

4 Model Embedding อย่างละเอียด (SignCLIP) 5

5 อัลกอริทึม Alignment อย่างละเอียด (คำอธิบายจากบทความ) 5

5.1 ฟังก์ชัน Cost . . . 5

5.2 ข้อจำกัด Locality และการ Normalize ด้วย Softmax . . . 5

5.3 การวนซ้ำ DP แบบ Global . . . 5

5.4 การกลั่นกรอง Sub-group . . . 5

5.5 Offset คงที่ก่อนและหลัง DP . . . 6

5.6 การค้นหาพารามิเตอร์แบบสุ่ม . . . 6

6 ชุดข้อมูลและ Baseline (จากบทความ) 6

II Workspace นี้ --- TSL Pipeline 8

7 เป้าหมายโครงการบนภาษามือไทย 8

7.1 สองภารกิจที่กำหนดสำหรับ TSL . . . 8

7.2 ตัวอย่าง TSL หนึ่งตัวที่ใช้ตลอดรายงาน . . . 8

8 การอธิบาย Pipeline ทีละขั้นตอน 8

8.1 ขั้นตอนที่ 0: Environment . . . 8

8.2 ขั้นตอนที่ 1: Extract CC จาก EAF ไปยัง WebVTT . . . 9

8.3 ขั้นตอนที่ 2: Pose Estimation . . . 9

8.4 ขั้นตอนที่ 3: Sign Segmentation . . . 10

8.5 ขั้นตอนที่ 4: Embedding (SignCLIP) . . . 10

8.6 เพิ่มเติม: สร้าง Gloss-text VTT . . . 10

8.7 ขั้นตอนที่ 5: Alignment ผ่าน Dynamic Programming . . . 11

8.7.1 ขั้นตอน 5.1 --- Pre-shift bias . . . 11

8.7.2 ขั้นตอน 5.2 --- หน้าต่าง Candidate . . . 11

8.7.3 ขั้นตอน 5.3 --- Similarity matrix . . . 11

8.7.4 ขั้นตอน 5.4 --- ฟังก์ชัน Cost . . . 11

8.7.5 ขั้นตอน 5.5 --- DP Recurrence . . . 11

8.7.6 ขั้นตอน 5.6 --- JIT Inner Loop . . . 11

8.7.7 Concrete invocation (B2) . . . 11

8.8 ขั้นตอนที่ 6: Post-processing . . . 12

8.8.1 การลบ cue ที่ทับซ้อน . . . 12

8.8.2 การรวม tier ต้นฉบับกลับเข้า EAF ที่อัปเดต . . . 12

8.8.3 การเพิ่ม output การทดลองทั้งหมดเป็น ELAN tier . . . 12

III ผลการทดลองจริง 13

9 การทดลอง 13

9.1 ตัวเลขหลัก (live, รันระหว่างการเขียนรายงาน) . . . 13

9.2 การอ่านผลลัพธ์ . . . 13

9.3 ผลลัพธ์ post-processing (การลบ overlap) . . . 14

9.4 ชุดพารามิเตอร์ที่ดีที่สุดบน TSL . . . 14

10 จุดที่ปรับได้ตลอด Pipeline ทั้งหมด 14

11 แผนที่โค้ด 14

12 Patch ที่ใช้กับ Upstream 14

12.1 SEA fork (vs. commit 5aaf27d) . . . 14

12.2 fairseq_signclip fork . . . 14

13 บทสรุปและขั้นตอนถัดไป 15

ภาค I

พื้นฐาน --- บทความ SEA

1 แรงจูงใจ: ทำไมต้องจัดตำแหน่งคำบรรยายภาษามือ?

การจัดตำแหน่งภาษามือ (Sign-language alignment) คือภารกิจในการนำคำบรรยายที่เป็นข้อความ--- ซึ่งเดิมผลิตขึ้นสำหรับเสียงของรายการ---และปรับ timestamp ให้ตรงกับช่วงเวลาที่ผู้แปลภาษามือแสดงท่าทางที่สอดคล้องกันจริงๆ บทความ SEA ต้นฉบับเปิดด้วยเหตุผลเชิงปฏิบัติสองประการที่สำคัญ

ข้อมูลคู่ขนานสำหรับการแปลภาษามือ ระบบการแปลภาษามือรุ่นปัจจุบัน [5, 6] ต้องการข้อมูลจำนวนมาก และ corpus ขนาดใหญ่ที่สุด---โดยเฉพาะ BOBSL (46 ชั่วโมงของรายการโทรทัศน์ BBC ที่มีล่ามภาษามือ)---มาพร้อมคำบรรยายที่ *จัดตำแหน่งตามเสียง* ไม่ใช่ *จัดตำแหน่งตามท่าทาง* ล่ามที่ต้องฟังก่อนแล้วค่อยแปลเป็นภาษามือมักล่าช้ากว่าเสียง 1--3 วินาที ดังนั้นคำบรรยายที่มีอยู่จึงคลาดเคลื่อนอย่างเป็นระบบ การจัดตำแหน่งใหม่เป็นเงื่อนไขเบื้องต้นสำหรับการฝึก model การแปล

การเข้าถึงและการบรรยายภายหลัง สำหรับผู้ชมที่หูหนวกที่ดู YouTube content ที่ผู้ใช้สร้างขึ้น การจัดตำแหน่งอัตโนมัติของไฟล์คำบรรยายที่มีสัญญาณรบกวนในรอบแรก จะลดความพยายามของมนุษย์ในการผลิตคำบรรยายที่ใช้งานได้อย่างมาก บทความ SEA อ้างถึงการวัดผลของ Bull et al. 2021 ที่ว่าผู้เชี่ยวชาญ ที่คล่องภาษามือต้องใช้เวลา 10--15 ชั่วโมงในการจัดตำแหน่งคำบรรยาย ต่อวิดีโอหนึ่งชั่วโมงด้วยมือ; ในอัตรา WMT-SLT 22 ที่ $40 ต่อชั่วโมง นั่นคือ $400--600 ต่อชั่วโมงของรายการ

ทำไมแนวทางก่อนหน้าจึงมีข้อจำกัด Model end-to-end สองตัวเป็น state of the art ก่อน SEA: *SAT* (Bull et al., 2021) และผู้สืบทอด SAT^+ (Jang et al., 2025) ทั้งสองเป็น transformer encoder ที่ fine-tune บนข้อมูล BSL/English ที่จัดตำแหน่งด้วยมือ $\sim$17.7 ชั่วโมง--- นั่นคือต้องการสิ่งที่พยายามผลิตอยู่แล้ว นอกจากนี้ยังทำนายในระดับ frame ภายในหน้าต่าง 20 วินาที และแก้ไขความขัดแย้งระหว่างหน้าต่างด้วย DTW ซึ่งเป็นแบบ local โดยพื้นฐาน

ข้อเสนอของ SEA SEA นำเสนอเป็นทางเลือกที่ *ไม่ต้องฝึก, ใช้ได้กับทุกภาษา* สมมติว่ามี model ภาษามือที่ฝึกมาแล้วสองตัว (segmenter และ embedder) และแทนที่ trained-aligner ด้วย dynamic programme แบบ global แบบไม่มี gradient บน *ทั้ง episode* การแลกเปลี่ยน: การค้นหาพารามิเตอร์เล็กน้อยต่อ dataset แลกกับการไม่ต้องการ supervision การจัดตำแหน่งแบบ in-domain และ pipeline แบบ modular

2 นิยามงานอย่างเป็นทางการและ Pipeline สามขั้นตอน

2.1 Input และ Output

ที่ให้มา วิดีโอภาษามือต่อเนื่องและลำดับหน่วยคำบรรยาย $t_1, \ldots, t_n$ แต่ละอันมีเวลาเริ่มต้นและสิ้นสุด (ที่อาจมีเสียงรบกวน) $\big(\text{start}(t_i), \text{end}(t_i)\big)$

เป้าหมาย ปรับ timestamp เริ่มต้น/สิ้นสุดของทุก t_i ให้ตรงกับการแสดงท่าทางมากขึ้น

2.2 สามขั้นตอนแบบ modular

SEA แบ่งปัญหาออกเป็นสามขั้นตอนแบบ plug-and-play; สองขั้นแรกเป็น model ที่ฝึกมาแล้ว ขั้นที่สามเป็นงานใหม่

1. **Segment** frames วิดีโอต่อเนื่องเป็นท่าทางแต่ละท่า $s_1, \ldots, s_m$ พร้อม $\big(\text{start}(s_j), \text{end}(s_j)\big)$ คล้ายกับการแบ่ง token ข้อความแต่ไม่มี vocabulary มาตรฐาน

2. **Embed** ท่าทางแต่ละ s_j และหน่วยคำบรรยายแต่ละ t_i เข้าสู่ space ที่ซ่อนร่วมกัน $\mathbb{R}^d$ ความคล้ายคลึง cross-modal $\langle u_i, v_j \rangle$ จึงมีความหมาย

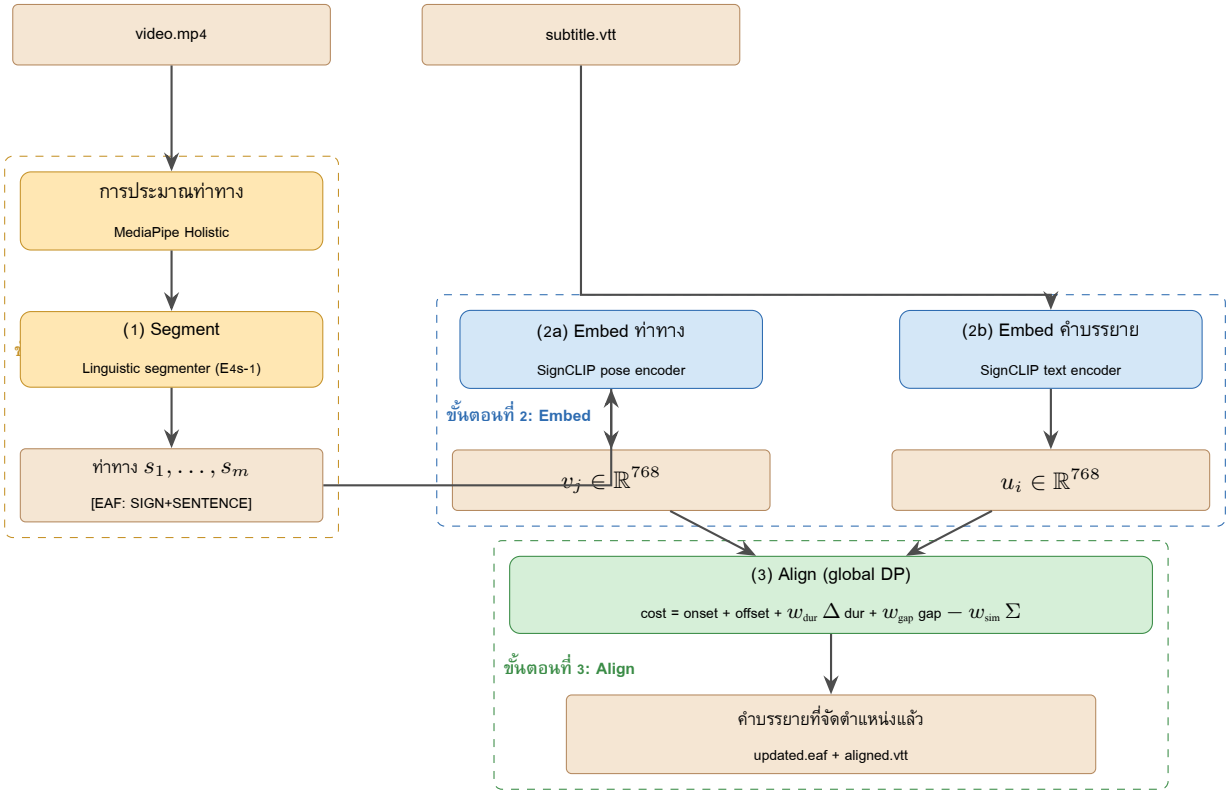
3. **Align**: เลือก สำหรับทุก t_i, ช่วงที่ *ต่อเนื่อง* $\{s_l, \ldots, s_r\}$ ที่ลด global cost รวมเทอมการจัดตำแหน่ง/prosodic และ (เพิ่มเติม) ความคล้ายคลึงเชิงความหมายที่ให้โดย embedding; เขียน timestamp ของ t_i ใหม่ตามขอบเขตของช่วงที่เลือก

บทความแยกตัวแปรอัลกอริทึมสองตัว: **Segment and Align** (เฉพาะ timing) และ **Segment, Embed, and Align** (timing + semantics) ทั้งคู่ใช้การวนซ้ำ DP เดียวกัน; ตัวที่สองเพิ่มเทอมพิเศษหนึ่งเทอมใน cost

2.3 ทำไมถึงใช้งานได้เป็น universal recipe

บทความนำเสนอสามจุดสถาปัตยกรรมที่ควรอ้างถึง

- Model *segmentation* ของท่าทางที่ฝึกบนภาษามือหนึ่งสามารถถ่ายโอน แบบ zero-shot ไปยังภาษาอื่น เพราะจำแนก primitive การเคลื่อนไหว ไม่ใช่ความหมาย Moryossef et al. 2023 รายงาน ROC-AUC 0.76 บน French Sign Language แม้ฝึกบน DGS



รูปที่ 1. สถาปัตยกรรม SEA สามขั้นตอนตามที่อธิบายในบทความ กล่องสีเหลืองคือ pipeline segmentation (ฝึกมาแล้ว), กล่องสีน้ำเงินคือ SignCLIP encoders (ฝึกมาแล้ว, dual-tower), และกล่องสีเขียวคืองานใหม่: DP แบบ global ที่เลือกช่วงท่าทางต่อเนื่องสำหรับทุกคำบรรยาย

- SignCLIP, model embedding, ฝึกแบบ *multilingual* บน SpreadtheSign (41 ภาษามือบนฝั่งวิดีโอ; อังกฤษบนฝั่งข้อความ) การ finetune เฉพาะภาษาให้ผลดีขึ้น-เมื่อมีข้อมูล in-domain แต่ไม่จำเป็น
- DP *global* optimum เข้าถึงทุกท่าทางใน episode ไม่ใช่แค่หน้าต่าง 20 วินาที ดังนั้นจึงตัดสินใจสม่ำเสมอ และไม่ต้องการการประมวลผลหลัง DTW แยกต่าง-หาก

3 Model Segmentation อย่างละเอียด

บทความใช้ model segmentation ทางภาษาศาสตร์ของ Moryossef et al. 2023 (``*Linguistically Motivated Sign Language Segmentation*'') ฝึกบนข้อมูล DGS ที่มี annotation ประมาณ ∼73 ชั่วโมงจาก Public DGS Corpus, รับ pose จาก MediaPipe Holistic และส่ง BIO tag ต่อ frame (Beginning, Inside, Outside) ผ่าน bidirectional LSTM ขนาดเล็ก checkpoint เฉพาะที่ใช้คือ model_E4s-1.pth (E4s-1), พร้อมพารามิเตอร์ scalar สองตัวที่แปลง probability ต่อ frame เป็น span discrete:

b-threshold **(ช่วง 0--100)**
: Probability ที่เหนือกว่านี้ ถือว่า frame เป็นขอบเขตระหว่างท่าทางสองท่าที่อยู่ติดกัน ค่าต่ำทำให้ segmentation ละเอียดขึ้น บทความพบค่าที่ดีที่สุดที่ 30 (BOBSL) และ 20--40 สำหรับ dataset อื่น

o-threshold **(ช่วง 0--100)**
: Probability ที่เหนือกว่านี้ ถือว่า frame อยู่นอกการแสดงท่าทาง (การหยุดพัก) บทความใช้ 30--50

ทำไมถึงใช้ segmenter นี้แทนตัวใหม่กว่า? บทความเปรียบเทียบทางเลือกหลายอย่างใน ablation study (ตาราง 4): model ที่ฝึกบน BSL-Corpus ของ Renz et al. 2021a/b บรรลุ F1 *segmentation* สูงกว่าบน BSLCP (47.71 vs. 31.13) แต่ F1 alignment *ต่ำกว่า* บน BOBSL (57.98 vs. 66.24) บทความให้เหตุผลว่า (i) ช่องว่าง domain ระหว่าง BSLCP และ BOBSL (ii) แนวโน้มของ Renz et al. ที่จะสร้าง signing-positive บน frame ที่ไม่ใช่การแสดงท่าทาง และ (iii) ความจริงที่ว่า alignment ปลายน้ำดูเหมือนจะได้ประโยชน์จาก *over-segmented เล็กน้อย* SEA จึงยึดใช้ model DGS ของ Moryossef

รายละเอียด implementation ที่เกี่ยวข้องกับ workspace นี้ CLI ที่ใช้คือ pose_to_segments, แจกจ่ายจาก github.com/J22Melody/segmentation@bsl เขียนไฟล์ ELAN .eaf ที่มีสอง tier: SIGN (annotation ต่อท่าทาง) และ SENTENCE (กลุ่มท่าทางที่อยู่ติดกันซึ่งก่อตัวเป็นวลี)

4 Model Embedding อย่างละเอียด (SignCLIP)

SignCLIP (Jiang et al., EMNLP 2024) คือ dual encoder แบบ CLIP สาขา text เป็น transformer แบบ BERT; สาขาท่าทางเริ่มต้นจาก BERT weights เดียวกันและปรับให้รับลำดับ frame pose จาก MediaPipe แทน sub-word ทั้งสองสาขาส่ง vector L2-normalized ขนาด 768 มิติ และฝึกด้วย contrastive InfoNCE loss ระหว่างคู่ที่จัดตำแหน่งแล้ว

Pretraining (*SignCLIP-multilingual*) checkpoint พื้นฐานที่ SEA ใช้คือ baseline_temporal (codename E7.2), pretrain บน SpreadtheSign---41 ภาษามือฝั่งวิดีโอ; อังกฤษฝั่งข้อความ---ดังนั้นจึงมีแบบแผน language-tag prepending <en> <bfi> (BSL/English), <en> <ase> (ASL/English), <de> <sgg> (DSGS/German)

การ Finetune เฉพาะภาษา บทความผลิต checkpoint เพิ่มเติมสามตัวโดย finetune *SignCLIP-multilingual* บน corpus เฉพาะภาษาต่างกัน:

- *SignCLIP-BSL*: ∼3.5 ล้าน sign spottings จาก BOBSL (Momeni et al., 2022), ใช้ text objective แบบ contrastive บน 8,697 sign ที่แตกต่างกัน
- *SignCLIP-ASL*: ∼200k ตัวอย่างรวมจาก PopSign (Starner et al., 2023), ASL Citizen (Desai et al., 2023), และ Sem-Lex (Kezar et al., 2023)
- *SignCLIP-Suisse*: 16,213 รายการศัพท์จาก Signsuisse แต่ละรายการจับคู่กับประโยคตัวอย่าง

ความสัมพันธ์ ISLR ↔ alignment บทความสังเกตความสัมพันธ์เชิงบวกที่สม่ำเสมอระหว่างคะแนน isolated-sign-language-recognition (ISLR) ของ embedding และ F1 alignment ปลายน้ำ สำหรับ BSL: SignCLIP-multilingual ได้ ISLR 0.5 และ F1@0.50 ที่ 66.70; SignCLIP-BSL ได้ ISLR 43.0 และ F1@0.50 ที่ 72.78

5 อัลกอริทึม Alignment อย่างละเอียด (คำอธิบายจากบทความ)

5.1 ฟังก์ชัน Cost

สำหรับการกำหนดผู้สมัคร cue t_i ให้กับช่วงท่าทางต่อเนื่อง $\{s_l, \ldots, s_r\}$, บทความนิยาม cost ต่อคู่ดังนี้

$$\begin{aligned}\phi(t_i,\, s_{l:r}) \;&=\; \big|\text{start}(t_i) - \text{start}(s_l)\big| + \big|\text{end}(t_i) - \text{end}(s_r)\big| \\ &\quad + w_{\text{dur}}\,\big|\,\text{dur}(t_i) - \text{dur}(s_{l:r})\,\big| + w_{\text{gap}}\,G(l,r) \\ &\quad - w_{\text{sim}}\,\Sigma(i;\, l, r),\end{aligned} \tag{1}$$

โดยที่เทอม inter-sign-gap คือ

$$G(l,r) \;=\; \sum_{j=l}^{r-1} \max\big(0,\; \text{start}(s_{j+1}) - \text{end}(s_j)\big),$$

และเทอม cumulative similarity คือ $\Sigma(i;\, l, r) \;=\; \sum_{j=l}^{r} M_{ij}$ โดยที่ M คือ similarity matrix (masked locality, normalized row-softmax) การตั้งค่า $w_{\text{sim}} = 0$ ได้ variant *Segment and Align* แบบ timing-only; เมื่อ $w_{\text{sim}} > 0$ จะได้ *Segment, Embed, and Align*

5.2 ข้อจำกัด Locality และการ Normalize ด้วย Softmax

สำหรับทุก cue row i, บทความเก็บเฉพาะ win_size $= 50$ ท่าทางที่ใกล้เคียงเชิงเวลาที่สุด (อื่น mask เป็น 0) จากนั้น row-softmax-normalize ด้วยอุณหภูมิ $\tau = 10$ ผลลัพธ์คือ ``matrix ของค่าที่กระจุกตัวรอบ diagonal เชิงเวลา" (บทความ, §A.2)

5.3 การวนซ้ำ DP แบบ Global

ตาราง DP คือ

$$\text{dp}[i,j] \;=\; \min_{i-1 \le k < j} \text{dp}[i-1,k] + \phi(t_i,\, s_{k:j-1}), \quad \text{dp}[0,0] = 0,\; \text{dp}[0, j>0] = \infty.$$

อัลกอริทึม *ไม่* กำหนดให้ cue สุดท้ายต้องลงบน sign สุดท้าย หลังจากเติมตาราง จุดสิ้นสุดที่ดีที่สุดคือ $j^\star = \arg\min_{0 \le j \le m} \text{dp}[n,j]$ และ backtracking กู้คืน assignment

5.4 การกลั่นกรอง Sub-group

หลังจาก DP แบบ global เลือกขอบเขต span $[l_i, r_i]$ สำหรับแต่ละ cue การ post-processing แบบ local แบ่ง span เป็น sub-span สูงสุด ที่ช่องว่างภายในทั้งหมด $\le$ max_gap จากนั้นเลือก sub-span ที่มี local cost ต่ำที่สุด

5.5 Offset คงที่ก่อนและหลัง DP

ทั้งก่อน *และ* หลัง DP, ทุก cue ถูก shift ด้วยคู่ offset คงที่ (δ_s, δ_e) ที่ได้จาก random search บทความสังเกตว่า ``offset 1 วินาทีคงที่ที่ใช้หลัง DP เป็นประโยชน์-เสมอใน dataset ที่ประเมิน"

5.6 การค้นหาพารามิเตอร์แบบสุ่ม

สำหรับแต่ละ dataset ที่มี training set บทความรัน combinations สุ่มมากถึง $10{,}000$ ครั้งบน CPU 128 ตัว ($\sim$1 วันของการค้นหา)

พารามิเตอร์	BOBSL	How2Sign	WMT-SLT	SwissSLi
Start offset ก่อน DP	2.6	1.3	1.4	0.49
End offset ก่อน DP	2.1	1.5	1.6	0.56
Start offset หลัง DP	0.0	0.0	0	0
End offset หลัง DP	1.0	1.0	1	1
b-threshold segmentation	30	40	20	20
o-threshold segmentation	50	50	30	30
win_size	50	50	50	50
max_gap (วินาที)	10	8	6	6
w_{dur}	1	5	0.5	0.5
w_{gap}	5	0.8	5	5
w_{sim}	10	10	5	1

ตารางที่ 1. การตั้งค่าพารามิเตอร์ที่ดีที่สุดต่อ dataset ที่รายงานในบทความ SEA (Appendix ตาราง 5) offset และ gap เป็นวินาที

6 ชุดข้อมูลและ Baseline (จากบทความ)

ชุดข้อมูล	ภาษา / แหล่งที่มา	Episodes	คำบรรยาย	ชั่วโมง	Offset (วินาที)
BOBSL	BSL/อังกฤษ, รายการ BBC ผสม (ล่าม)	55	31,479	$\sim$46	2.70/2.70
How2Sign	ASL/อังกฤษ, บันทึก studio re-signing	2,528	35,191	$\sim$80	1.57/2.13
WMT-SLT SRF	DSGS/เยอรมัน, ข่าว live interpretation	29	7,071	$\sim$16	1.06/1.75
SwissSLi mit.	DSGS/เยอรมัน, รายการแปลโดยคนหูหนวก	14	554	$\sim$1	0.49/0.56

ตารางที่ 2. ชุดข้อมูลภาษามือสี่ชุดในการประเมิน SEA (ตารางบทความ 1) Offset คือการ shift ค่ามัธยฐาน (หรือค่าเฉลี่ยสำหรับ BOBSL) ระหว่างคำบรรยายที่จัดตำแหน่ง-ตามเสียงและแบบมนุษย์

เมตริกการประเมิน: F1@0.50 คำบรรยายที่ทำนายนับว่าถูกต้องเมื่อ segment ที่ทำนายทับซ้อนกับ ground truth ด้วย intersection-over-union (IoU) ≥ 0.50

ผลลัพธ์หลัก (ตารางบทความ 2) SEA เป็น SOTA บนทุก test set

Pseudocode (Appendix Figure 4 ของบทความ)

```
# ขั้นตอนที่1 □□ Segment
signs = segmenter(video)
# ขั้นตอนที่2 □□ Embed
v_emb = embed_sign(signs)
t_emb = embed_text(subtitles)
sim_matrix = dot_product(t_emb, v_emb)
# ขั้นตอนที่3 □□ Align
def cost(cue, span):
    a = abs(cue.start □ span.start)
    b = abs(cue.end □ span.end)
```

	BOBSL Test	How2Sign Test	WMT-SLT Test	SwissSLi Test
Original	14.11	33.06	46.85	60.48
Original^{+}	44.61	36.21	74.83	---
SAT	54.57	---	75.32	---
SAT^{+}	63.81	---	---	---
Segment and Align	49.58	36.17	76.83	84.19
SEA (multilingual)	50.68	37.51	76.43	**85.57**
SEA (lang-finetuned)	54.50	**39.57**	**77.69**	85.22
SignCLIP-BSL + SAT^{+}	**65.68**	---	---	---

ตารางที่ 3. F1@0.50 บนชุดข้อมูล test สี่ชุด, ตารางบทความ 2 (เฉพาะคอลัมน์ test) SEA เป็น SOTA บนทุก test set

```
        c = abs(cue.duration □ span.duration)
        d = compute_internal_gap(span)
        e = compute_total_similarity(cue, span)
        return a + b + w_dur*c + w_gap*d □ w_sim*e

sim_matrix = zero_out(sim_matrix, win_size)
sim_matrix = softmax_normalize(sim_matrix)

dp = aligner(signs, subtitles, cost, sim_matrix)
bounds = backtrack(dp)

# การกลั่นกรองinternal gap
for i in range(len(subtitles)):
        group = signs[bounds[i]:bounds[i+1]]
        subgroups = split_on_gap(group, max_gap)
        alignments[i] = pick_best(subgroups, subtitles[i], cost)

# อัปเดตtimestamp
for i in range(len(subtitles)):
        subtitles[i].start = alignments[i][0].start
        subtitles[i].end = alignments[i][□1].end
```

ภาค II

Workspace นี้ --- TSL Pipeline

7 เป้าหมายโครงการบนภาษามือไทย

Workspace นี้ปรับ SEA ให้ใช้กับภาษามือไทย (TSL) codebase เป็น fork ของ J22Melody/SEA (commit 5aaf27d) บวก SignCLIP encoder จาก J22Melody/fairseq, พร้อม patch เล็กน้อยที่จำเป็นสำหรับ Windows และ multi-model live embedding (§12)

7.1 สองภารกิจที่กำหนดสำหรับ TSL

ภารกิจที่ 1 (จุดสนใจของรายงานนี้) นำ CC cue จำนวน 172 รายการที่มี timestamp จัดตำแหน่งตามเสียง และผลิต timestamp จัดตำแหน่งตามภาพที่ตรงกับ annotation ผู้เชี่ยวชาญ 119 รายการ (CC_Aligned) ที่ผลิตโดยนักวิจัย

ภารกิจที่ 2 (งานในอนาคต) ขยาย annotation Gloss ระดับประโยค (119 รายการ) เป็น annotation ต่อท่าทาง 852 รายการ Gloss_Labeling โดยอัตโนมัติ

7.2 ตัวอย่าง TSL หนึ่งตัวที่ใช้ตลอดรายงาน

วิดีโอบรรยาย *``การเปรียบเทียบและเรียงลำดับ''* 11 นาที 07 วินาที, 1920×1080@60fps ทั้งสี่ tier ground-truth ใน EAF คือ:

Tier	เนื้อหา	จำนวน
CC	คำบรรยายภาษาไทยดิบ จัดตำแหน่งตามเสียง (input)	172
CC_Aligned	คำบรรยายที่จัดตำแหน่งใหม่ด้วยมือ *(GT)*	119
Gloss	สตริง gloss ภาษามือ ระดับประโยค	119
Gloss Labeling	Annotation gloss ต่อท่าทาง	852

8 การอธิบาย Pipeline ทีละขั้นตอน

สำหรับแต่ละขั้นตอนระบุ **(a)** *ขั้นตอนนี้คืออะไร*, **(b)** *ทำงานอย่างไรใน Workspace นี้* (commands, พารามิเตอร์, อ้างอิงโค้ด), **(c)** *รายละเอียดอัลกอริทึม* ที่ควรทราบ, และ **(d)** *artefact ที่ขั้นตอนนี้ผลิต*

8.1 ขั้นตอนที่ 0: Environment

คืออะไร Python 3.11 virtual environment พร้อม PyTorch (CUDA 12.8 สำหรับ NVIDIA RTX 5060 Ti ในเครื่อง, Blackwell, sm_120), MediaPipe ที่ pin ที่ 0.10.21, BSL-branch sign segmenter, บวกไลบรารี subtitle/EAF มาตรฐาน

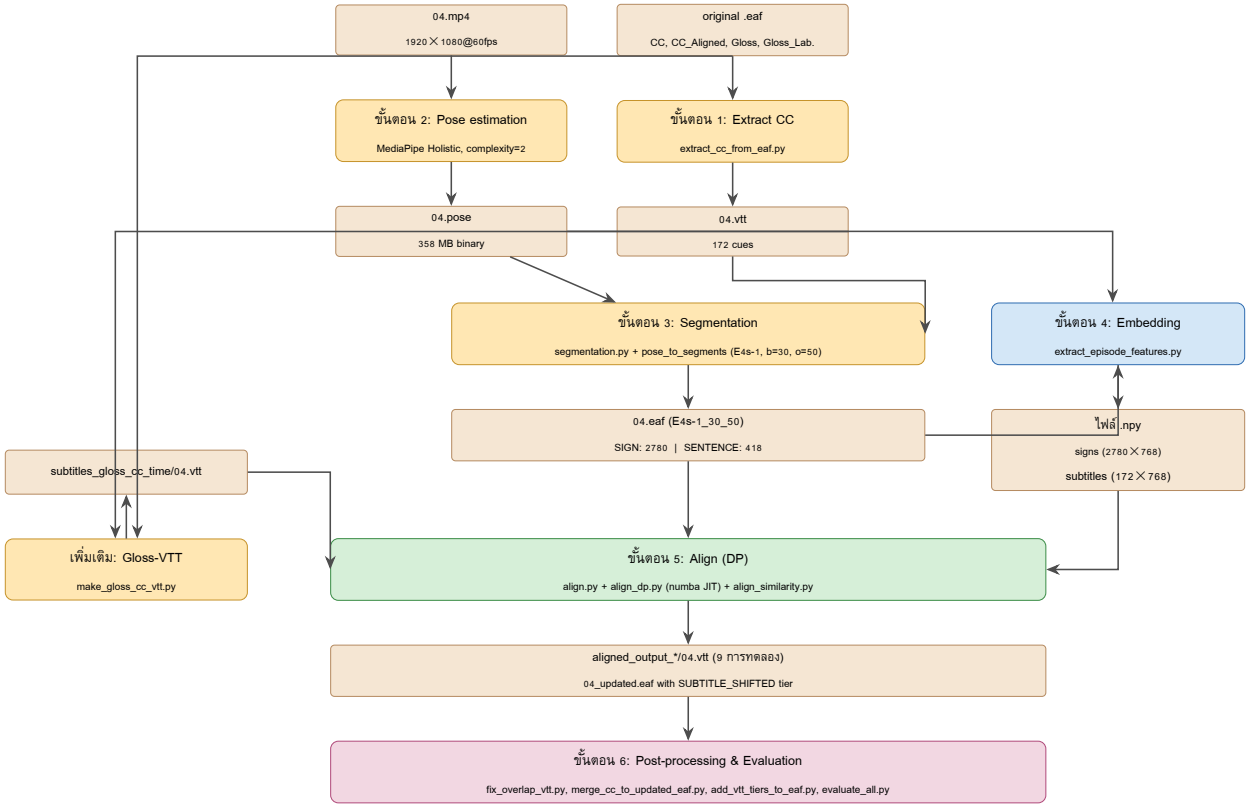
Hardware Intel Core Ultra 7 265K (20 cores), RAM 64 GB, RTX 5060 Ti พร้อม VRAM 17.1 GB, Windows 11

การติดตั้ง (ครั้งเดียว)

```
python □m venv venv
venv\Scripts\activate
pip install pysrt webvtt□py lxml numpy pympi□ling
pip install "mediapipe==0.10.21" pose□format
pip install beartype numba tqdm scikit□learn tabulate
pip install "git+https://github.com/J22Melody/segmentation@bsl"
pip install □ □force□reinstall torch \
    □ □index□url https://download.pytorch.org/whl/cu128
```

ปัญหาที่พบระหว่างการติดตั้ง

- **MediaPipe ต้องเป็น 0.10.21**: เวอร์ชัน 0.10.22 ขึ้นไปทำให้ videos_to_poses พัง
- **PyTorch** cu128 จำเป็นสำหรับ sm_120; cu126 จะ fallback ไป CPU อย่างเงียบๆ บน Blackwell



รูปที่ 2. End-to-end workspace pipeline สีเหลือง = การรับข้อมูล/segmentation; สีน้ำเงิน = SignCLIP embedding; สีเขียว = DP alignment แบบ global; สีชมพู = post-processing และ evaluation; สีครีม = artefact ข้อมูล (พร้อมขนาด/จำนวนจริงใน workspace) กล่องที่ระบุว่า "*เพิ่มเติม*" คือเครื่องมือที่พัฒนาใน workspace นี้ ไม่ใช่ส่วนของ SEA upstream

- numba จำเป็นสำหรับ JIT-compiled DP inner loop
- video_ids.txt ต้องเป็น UTF-8 ไม่มี BOM echo 04 > video_ids.txt บน Windows ผลิต UTF-16 BOM และทำให้ segmenter พัง

ผลลัพธ์ Environment ที่ reproducible ที่ SEA/venv/

8.2 ขั้นตอนที่ 1: Extract CC จาก EAF ไปยัง WebVTT

คืออะไร ไฟล์ annotation ภาษาไทยต้นฉบับอยู่ในรูปแบบ XML ของ ELAN SEA คาดว่า subtitle จะอยู่ใน WebVTT (หรือ SRT) ขั้นตอนนี้ parse tier CC จาก EAF และเขียน .vtt มาตรฐาน

โค้ด example_alignment/extract_cc_from_eaf.py

อัลกอริทึม

1. Parse EAF ด้วย xml.etree.ElementTree
2. สร้าง dictionary TIME_SLOT_ID ⟶ วินาที (EAF เก็บเวลาเป็นมิลลิวินาที)
3. ค้นหา TIER ที่ TIER_ID="CC", จากนั้นสำหรับทุก ANNOTATION อ่าน TIME_SLOT_REF1/REF2 และข้อความ ANNOTATION_VALUE
4. เรียง cue ตามเวลาเริ่มต้นและส่ง WebVTT มาตรฐาน

ผลลัพธ์ (ยืนยันแล้ว) 04.vtt ที่มีคำบรรยายพอดี **172** รายการ

8.3 ขั้นตอนที่ 2: Pose Estimation

คืออะไร แปลงวิดีโอ pixel-domain เป็นลำดับโครงกระดูกมนุษย์ MediaPipe Holistic ทำนาย 543 landmark ต่อ frame: 33 ร่างกาย, 21×2 มือ, และ 468 จุดใบหน้า

รัน

```
videos_to_poses □□format mediapipe □□directory . \
    □□additional□config="model_complexity=2,smooth_landmarks=false,refine_face_landmarks=true"
```

ทำไมถึงใช้การตั้งค่านี้

- model_complexity=2: ความแม่นยำสูงสุด ใช้เวลา CPU ประมาณ 15 นาทีต่อวิดีโอ 11 นาที 60fps
- smooth_landmarks=false: segmenter เองเป็น LSTM ที่ smooth ระหว่าง frame; การ pre-smooth จะลบขอบเขตออก
- refine_face_landmarks=true: TSL ใช้ non-manual ของใบหน้า/ริมฝีปากในระดับเดียวกับรูปมือ

ผลลัพธ์ (ยืนยันแล้ว) 04.pose (**358 MB** binary)

8.4 ขั้นตอนที่ 3: Sign Segmentation

คืออะไร ตัด pose stream ต่อเนื่องเป็นหน่วยท่าทางแต่ละท่า และจัดกลุ่มเป็นประโยค output เป็น EAF ที่มีสอง tier: SIGN (หนึ่งรายการต่อท่าทาง) และ SENTENCE (กลุ่มท่าทางต่อเนื่องที่ก่อตัวเป็นวลี)

Model model_E4s-1.pth (ส่วนที่ 3), ฝึกบน DGS แต่ถ่ายโอนแบบ zero-shot

รัน

```
cd SEA
python segmentation.py \
    □□sign□b□threshold 30 □□sign□o□threshold 50 □□num_workers 1 \
    □□video_ids ..\example_alignment\video_ids.txt \
    □□pose_dir ..\example_alignment \
    □□save_dir ..\example_alignment\segmentation_output \
    □□video_dir ..\example_alignment
```

Patch ใน Workspace

```
# ก่อนพังบน( Windows):
result = subprocess.run(cmd, shell=True)
# หลังทำงานได้บน( Windows):
result = subprocess.run(shlex.split(cmd), shell=False)
```

ผลลัพธ์ (ยืนยันแล้ว) segmentation_output/E4s-1_30_50/04.eaf ที่มี **2,780** รายการ SIGN และ **418** รายการ SENTENCE

8.5 ขั้นตอนที่ 4: Embedding (SignCLIP)

คืออะไร Embed ทุก segment SIGN เป็น vector 768 มิติ และ embed ทุก cue คำบรรยายเข้าสู่ space 768 มิติ *เดียวกัน* เพื่อให้เปรียบเทียบได้โดยตรงด้วย dot product

สาม checkpoint ที่ใช้ใน workspace

- bsl (⟶ bobsl_finetune): SignCLIP-multilingual ที่ finetune บน BOBSL sign spottings ∼3.5 ล้านรายการ
- multilingual (⟶ baseline_temporal): base ที่ pretrain บน SpreadtheSign
- asl (⟶ asl_finetune): finetune บน PopSign+ASL Citizen+Sem-Lex

ผลลัพธ์ (ยืนยันแล้ว)

8.6 เพิ่มเติม: สร้าง Gloss-text VTT

ทำไม text encoder ของ SignCLIP ไม่เคยฝึกบนประโยคภาษาไทยเต็มรูปแบบ แต่รู้จัก token gloss ของภาษามือ (ซึ่งคล้ายกับข้อมูล gloss-style ที่ encoder เห็นระหว่าง pretraining) การแทนที่ประโยคภาษาไทยด้วย gloss ภาษามือที่สอดคล้องกัน ทำให้ similarity matrix มีการแยกแยะดีขึ้น

ผลลัพธ์ ข้อความ CC *``เราจะเรียนเกี่ยวกับการเปรียบเทียบและการเรียงลำดับตัวเลข''* กลายเป็น gloss *``HELLO STUDENT NEXT WHERE LEARN COMPARISON ORDERING''* --- สั้น แบบ sign-vocabulary ตรงกับสิ่งที่ text encoder ของ SignCLIP ชอบ

ไฟล์	Variant	Shape ที่ยืนยัน
segmentation_embedding/sign_clip/04.npy	BSL signs	2780 × 768
segmentation_embedding/sign_clip_multi/04.npy	Multilingual signs	2780 × 768
segmentation_embedding/sign_clip_asl/04.npy	ASL signs	2780 × 768
subtitle_embedding/sign_clip/04.npy	BSL, CC text	172 × 768
subtitle_embedding/sign_clip_multi/04.npy	Multi, CC text	172 × 768
subtitle_embedding/sign_clip_multi_gloss/04.npy	Multi, Gloss text	172 × 768
subtitle_embedding/sign_clip_asl/04.npy	ASL, CC text	172 × 768
subtitle_embedding/sign_clip_asl_gloss/04.npy	ASL, Gloss text	172 × 768

ตารางที่ 4. Embedding artefact ทั้งหมดที่มีใน workspace นี้ Shape ยืนยันโดยการโหลดทุก .npy ด้วย numpy

8.7 ขั้นตอนที่ 5: Alignment ผ่าน Dynamic Programming

นี่คือหัวใจทางเทคนิคของ SEA Driver: SEA/align.py; DP recurrence: SEA/align_dp.py; similarity matrix: SEA/align_similarity.py

8.7.1 ขั้นตอน 5.1 — Pre-shift bias

SEA เริ่มต้นด้วยการ shift ทุก cue c_i ไปข้างหน้าในเวลา ด้วย (δ_s, δ_e) คงที่ สำหรับ BOBSL บทความพบ $(2.6, 2.1)$ วินาที สำหรับตัวอย่าง TSL workspace นี้ใช้:

- $(2.6, 2.1)$ วินาที สำหรับ profile CC-text มาตรฐาน (B1)
- $(1.8, 1.5)$ วินาที สำหรับ profile CC-text ที่ tuned (B2/B_MULTI/D_ASL)
- $(1.3, 1.0)$ วินาที สำหรับ profile Gloss-text (C_MULTI/*_gloss/*_word)

8.7.2 ขั้นตอน 5.2 — หน้าต่าง Candidate

สำหรับทุก cue c_i อัลกอริทึมจำกัดความสนใจไปที่ dp_window_size sign segments ที่จุดกึ่งกลางเชิงเวลาอยู่ใกล้ที่สุด (default 50; profile ที่ tuned ใช้ 40) สิ่งที่อยู่นอกหน้าต่างได้รับ cost อนันต์และ similarity 0 ข้อจำกัด locality นี้ bound run-time DP เป็นประมาณ $O(M \cdot W^2)$

8.7.3 ขั้นตอน 5.3 — Similarity matrix

ด้วย --similarity_measure sign_clip_embedding, raw similarity ระหว่าง cue i และ sign j คือ dot product ของ embedding L2-normalized สองตัว:

$$M^0_{ij} = \langle u_i, v_j \rangle, \quad u_i \in \mathbb{R}^{768},\ v_j \in \mathbb{R}^{768}.$$

8.7.4 ขั้นตอน 5.4 — ฟังก์ชัน Cost

เหมือนกับสมการ (1) ด้วย profile ที่ tuned ที่ C_MULTI, weights คือ $w_{\text{dur}}{=}2, w_{\text{gap}}{=}8, w_{\text{sim}}{=}6$, max_gap$=6$ วินาที, window_size$=40$

8.7.5 ขั้นตอน 5.5 — DP Recurrence

เหมือนกับส่วนที่ 5 ด้วย $M = 172$ และ $N = 2780$, ตารางมี 173×2781 รายการ backtracking เริ่มที่ $j^\star = \arg\min_j \text{dp}[M, j]$

8.7.6 ขั้นตอน 5.6 — JIT Inner Loop

align_dp.dp_inner_loop ถูก decorate ด้วย numba.@njit เมื่อเปิด JIT, episode 11 นาทีเสร็จในเวลาน้อยกว่าหนึ่งวินาที

8.7.7 Concrete invocation (B2)

```
python align.py □□overwrite □□mode=inference \
  □□video_ids ..\example_alignment\video_ids.txt □□num_workers 1 \
  □□dp_duration_penalty_weight 2 □□dp_gap_penalty_weight 8 \
  □□dp_max_gap 6 □□dp_window_size 40 \
  □□sign□b□threshold 30 □□sign□o□threshold 50 \
```

```
□□pr_subs_delta_bias_start 1.8 □□pr_subs_delta_bias_end 1.5 \
□□similarity_measure sign_clip_embedding □□similarity_weight 6 \
□□pr_sub_path ..\example_alignment\subtitles \
□□segmentation_dir ..\example_alignment\segmentation_output \
□□subtitle_embedding_dir ..\example_alignment\subtitle_embedding\sign_clip \
□□segmentation_embedding_dir ..\example_alignment\segmentation_embedding\sign_clip \
□□save_dir ..\example_alignment\aligned_output_with_embedding_tuned
```

8.8 ขั้นตอนที่ 6: Post-processing

8.8.1 การลบ cue ที่ทับซ้อน

output ของ SEA มักมี cue ที่ทับซ้อน: ประมาณ 88% ของคู่ที่อยู่ติดกันทับซ้อนกัน เพราะมี cue input 172 รายการแต่มีเพียง ∼119 slot ที่จัดตำแหน่งโดยมนุษย์ script fix_overlap_vtt.py clamp end ของแต่ละ cue เป็น start ของ cue ถัดไป:

$$\mathrm{end}(c_i) \;\leftarrow\; \min\Big(\mathrm{end}(c_i),\ \mathrm{start}(c_{i+1})\Big).$$

8.8.2 การรวม tier ต้นฉบับกลับเข้า EAF ที่อัปเดต

merge_cc_to_updated_eaf.py คัดลอกสี่ tier ต้นฉบับ (CC, CC_Aligned, Gloss, Gloss Labeling) กลับเข้า 04_updated.eaf

8.8.3 การเพิ่ม output การทดลองทั้งหมดเป็น ELAN tier

add_vtt_tiers_to_eaf.py รวม VTT output ที่จัดตำแหน่งแล้วทั้งหมด เข้าไฟล์ ELAN เดียว---*comparison EAF*---เพื่อให้นักวิจัย เปรียบเทียบภาพได้บน timeline เดียว

Tier	จำนวน annotation
CC	172
CC_Aligned (GT)	119
Gloss	119
Gloss Labeling	852
SUBTITLE_B2	172
SUBTITLE_B_MULTI	172
SUBTITLE_C_MULTI	172
SUBTITLE_C_MULTI_word	172
SUBTITLE_D_ASL	172
SUBTITLE_D_ASL_gloss	172
SUBTITLE_D_ASL_word	172

ตารางที่ 5. รายการ tier ที่ยืนยันแล้วของ comparison EAF ใน workspace นี้ การโหลดไฟล์ใน ELAN จะวาง 11 track ทั้งหมดบน timeline เดียว

ภาค III

ผลการทดลองจริง

9 การทดลอง

Workspace มีไดเรกทอรี output ที่จัดตำแหน่งแล้วเก้าไดเรกทอรี ดังแสดงในตาราง 6 ซึ่งต่างกันเฉพาะ embedding model (BSL / Multilingual / ASL), แหล่งข้อความที่ป้อนให้ SignCLIP (CC vs. Gloss), DP weights (profile *มาตรฐาน* vs. *tuned*), และว่าใช้ word-level tokenization หรือไม่

ID	คำอธิบาย	โฟลเดอร์	จำนวน cue
A	ไม่มี embedding, DP มาตรฐาน	aligned_output	172
B1	BSL emb., DP มาตรฐาน	aligned_output_with_embedding	172
B2	BSL emb., DP *tuned*	aligned_output_with_embedding_tuned	172
B_MULTI	Multilingual + CC text, B2 weights	aligned_output_multi_b2	172
C_MULTI	**Multilingual + Gloss text (ดีที่สุด)**	aligned_output_multi_gloss	172
C_MULTI_word	Multilingual + Gloss + word-level	aligned_output_multi_gloss_word	172
D_ASL	ASL + CC text	aligned_output_asl_b2	172
D_ASL_gloss	ASL + Gloss text	aligned_output_asl_gloss	172
D_ASL_word	ASL + Gloss + word-level	aligned_output_asl_gloss_word	172

ตารางที่ 6. การทดลองเก้าชุดที่ดำเนินการใน workspace นี้ จำนวน cue ยืนยันโดยการนับ --> markers ในแต่ละ VTT; ทุก output รักษา cardinality ของ input ที่ 172

9.1 ตัวเลขหลัก (live, รันระหว่างการเขียนรายงาน)

การทดลอง	ค่าเฉลี่ย	มัธยฐาน	S.D.	± 1 วินาที	± 2 วินาที	± 3 วินาที	ทับซ้อน
B2 (BSL, CC text, tuned)	$+1.02$ s	-0.05 s	5.53 s	74 %	96 %	97 %	88.3 %
B_MULTI (multi, CC text)	$+0.91$ s	-0.05 s	5.50 s	78 %	**97 %**	**99 %**	88.3 %
C_MULTI (multi, Gloss)	$+0.49$ s	-0.15 s	5.54 s	**80 %**	96 %	**99 %**	**87.7 %**
C_MULTI_word	$+0.51$ s	-0.15 s	5.48 s	77 %	96 %	99 %	88.3 %
D_ASL (ASL, CC text)	$+1.25$ s	$+0.37$ s	5.54 s	59 %	81 %	96 %	88.9 %
D_ASL_gloss	$+0.77$ s	-0.10 s	5.50 s	64 %	91 %	97 %	88.3 %
D_ASL_word	$+0.78$ s	-0.11 s	5.50 s	67 %	93 %	96 %	87.7 %
CC_Aligned (GT)	0 s	0 s	0 s	100 %	100 %	100 %	**0.0 %**

ตารางที่ 7. คุณภาพ alignment บนวิดีโอ TSL ทดสอบ (รัน 2026-04-25) ค่าเฉลี่ย/มัธยฐาน/S.D. คือ offset เวลาเริ่มต้น signed vs. CC_Aligned; บวก = SEA วาง caption ช้ากว่ามนุษย์ ทับซ้อน คือสัดส่วนของคู่ cue ที่อยู่ติดกันที่ทับซ้อน *ก่อน* ที่จะใช้ fix_overlap_vtt.py

9.2 การอ่านผลลัพธ์

- **SignCLIP Multilingual ดีกว่าทั้ง BSL และ ASL บน TSL** แม้ว่า multilingual base จะไม่เคยฝึกบนข้อมูล TSL, การครอบคลุม 41 ภาษาบนฝั่งวิดีโอและ text decoder ภาษาอังกฤษ ถ่ายโอนได้ดีกว่า variant เฉพาะภาษาทั้งสอง
- **Gloss text ดีกว่า CC text มาก** การแทนที่ประโยคภาษาไทยด้วย gloss ที่สอดคล้องกัน ลด mean offset จาก $+0.91$ s เป็น $+0.49$ s และเพิ่ม coverage ± 1 s จาก 78% เป็น 80%
- **Word-level pooling ไม่มีประโยชน์ที่สม่ำเสมอบนวิดีโอนี้** C_MULTI_word และ D_ASL_word ทำงานได้ภายใน noise ของคู่ระดับประโยค
- **Overlap rate เป็น structural** 172 cue input แต่มีเพียง $\sim$119 slot ที่แตกต่างใน ground truth $\Rightarrow$ ทุก variant ผลิต overlap $\sim$88% ซึ่งแก้ไข ด้วย fix_overlap_vtt.py ในการแสดงผล

9.3 ผลลัพธ์ post-processing (การลบ overlap)

ไฟล์	ก่อน	หลัง
aligned_output_multi_gloss/04.vtt	150/171 (87.7 %)	**0/171 (0.0 %)**
aligned_output_asl_gloss/04.vtt	151/171 (88.3 %)	**0/171 (0.0 %)**

ตารางที่ 8. Overlap ก่อนและหลัง clamping pass เชิงสำอาง ไฟล์ 04_no_overlap.vtt ที่สอดคล้องกันมีอยู่ใน workspace และแนะนำสำหรับการแสดงผลต่อมนุษย์ ไม่ใช่การประเมิน

9.4 ชุดพารามิเตอร์ที่ดีที่สุดบน TSL

พารามิเตอร์	ดีที่สุดบน TSL (C_MULTI)
--similarity_measure	sign_clip_embedding
--similarity_weight	6
--dp_duration_penalty_weight	2
--dp_gap_penalty_weight	8
--dp_window_size	40
--dp_max_gap	6
--sign-b-threshold	30
--sign-o-threshold	50
--pr_subs_delta_bias_start	1.3
--pr_subs_delta_bias_end	1.0
--post_subs_delta_bias_start	0.0
--post_subs_delta_bias_end	1.0
SignCLIP variant	multilingual
Subtitle text	subtitles_gloss_cc_time/04.vtt

ตารางที่ 9. ชุดพารามิเตอร์ที่ผลิต mean offset ต่ำที่สุดบนวิดีโอ TSL ทดสอบ ($+0.49$ s)

10 จุดที่ปรับได้ตลอด Pipeline ทั้งหมด

11 แผนที่โค้ด

12 Patch ที่ใช้กับ Upstream

12.1 SEA fork (vs. commit 5aaf27d)

- SEA/align.py: เพิ่ม multi-model live embedding ผ่าน --live_model_name, --live_language_tag; การโหลด segmentation embeddings ที่คำนวณล่วงหน้า; dynamic path resolution
- SEA/align_dp.py: เพิ่ม graceful fallback ที่แทนที่ numba.@njit ด้วย identity decorator เมื่อ numba หรือ LLVM ไม่สามารถ import ได้
- SEA/segmentation.py: video paths ถูก wrap ด้วย os.path.abspath, และ subprocess.run ถูก invoke ด้วย shlex.split(cmd) และ shell=False สำหรับ Windows

12.2 fairseq_signclip fork

ไฟล์หลายรายการได้รับการแก้ไข Windows-path / model-loading:

- mmpt/models/mmfusion.py
- mmpt/processors/dsprocessor.py

ขั้นตอน	Flag	บทบาท	ผลกระทบ
Pose estimation	model_complexity	ความแม่นยำ/ความเร็ว MediaPipe	สูงขึ้น = ช้าลง, landmark ดีขึ้น
Segmentation	--sign-b-threshold	cutoff ความน่าจะเป็น boundary	สูง $\Rightarrow$ segment น้อยลง
Segmentation	--sign-o-threshold	cutoff ``outside signing''	สูง $\Rightarrow$ รักษาการเคลื่อนไหวอ่อน
Pre-DP shift	--pr_subs_delta_bias_*	bias audio$\rightarrow$visual หยาบ	ตั้งตาม median offset ของ GT
Embedding	--live_model_name	SignCLIP variant	multi / BSL / ASL
Embedding text	--pr_sub_path	CC vs. Gloss VTT	Gloss ดีกว่าบน TSL
DP cost	--dp_duration_penalty_weight	w_{dur}	สูง $\Rightarrow$ match duration แน่น
DP cost	--dp_gap_penalty_weight	w_{gap}	สูง $\Rightarrow$ หลีก span ที่มีช่องว่าง
DP cost	--similarity_weight	w_{sim}	สูง $\Rightarrow$ เชื่อ embedding
Post-DP shift	--post_subs_delta_bias_*	padding end เพิ่มเติม	1 s end pad ``เป็นประโยชน์เสมอ''
Post-process	fix_overlap_vtt.py	clamp เวลาสิ้นสุด	88 % $\rightarrow$ 0 % overlap (การแสดงผล)

ตารางที่ 10. ทุกจุดที่ปรับได้ใน SEA pipeline ตามที่ใช้งานใน workspace นี้

ไฟล์	บทบาท
SEA/segmentation.py	Wrapper สำหรับ pose_to_segments; วนซ้ำบน grid พารามิเตอร์ (model, b, o); ผลิต EAF ของ SIGN/SENTENCE
SEA/align.py	Driver pipeline: โหลด cue + signs, รัน similarity, dispatch DP, เขียน VTT/EAF
SEA/align_dp.py	Numba-JIT DP recurrence + sub-group refinement post-pass
SEA/align_similarity.py	คำนวณ similarity matrix หลายแบบ พร้อม normalization
SEA/config.py	Argparse layer; flag CLI ทั้งหมด
SEA/utils.py	Subtitle I/O (VTT/SRT), shifting, EAF I/O, normalization helpers
example_alignment/extract_cc_from_eaf.py	EAF $\rightarrow$ VTT extractor (ขั้นตอน 1)
example_alignment/make_gloss_cc_vtt.py	Gloss-text VTT builder (ส่วนที่ 8.6)
example_alignment/fix_overlap_vtt.py	End-time clamping overlap fixer
example_alignment/evaluate_all.py	คำนวณ metrics ในตาราง 7

ตารางที่ 11. ตำแหน่งของแต่ละส่วนของ pipeline ใน workspace นี้

- mmpt/processors/dsprocessor_sign.py
- mmpt/processors/processor.py
- mmpt/tasks/task.py
- scripts_bsl/extract_episode_features.py

13 บทสรุปและขั้นตอนถัดไป

สิ่งที่ทำงานได้บน TSL แม้ SEA ออกแบบและประเมินบน BSL/ASL/DSGS, pipeline ถ่ายโอนไปยัง TSL ได้อย่างสะดวกโดยไม่ต้องฝึกใหม่: 80% ของ cue ที่ match ตกภายใน ± 1 วินาที จาก ground truth ที่จัดตำแหน่งโดยมนุษย์ และ 99% ภายใน ± 3 วินาที เมื่อใช้ SignCLIP Multilingual บน Gloss text stack ที่ใช้ pose (MediaPipe $\rightarrow$ Moryossef segmenter $\rightarrow$ SignCLIP multilingual) เป็น language-agnostic อย่างแท้จริง

สิ่งที่ยังทำงานไม่ได้

- SignCLIP ที่ finetune บน ASL ถ่ายโอนได้ *แย่กว่า* baseline multilingual--- ผลลัพธ์เชิงลบที่มีประโยชน์
- Overlap rate 88% เป็น structural: อัลกอริทึมไม่สามารถลบหรือรวม cue ได้
- ตัวเลข F1@0.50 ในบทความ SEA ไม่สามารถเปรียบเทียบโดยตรงเพราะ ground truth TSL มีรายการน้อยกว่า set ที่ทำนาย (119 vs. 172)

ขั้นตอนถัดไปที่แนะนำ

1. **TSL-specific finetuning ของ SignCLIP** บน TSL lexicon ขนาดเล็ก (เช่น พจนานุกรม TSL สำหรับโรงเรียน)

2. **การรวม cue** เป็น post-processing pass ของ SEA: เมื่อ output SEA สองรายการที่อยู่ติดกัน map ไปยัง span ที่ทับซ้อน ให้รวมเป็น cue เดียวแทนการ clamp
3. **Implement ภารกิจที่ 2** (Gloss $\longrightarrow$ Gloss Labeling): ใช้ token-level similarity path ที่มีอยู่และจำกัด DP ให้ทำงาน ภายในขอบเขตเวลาของแต่ละ Gloss sentence
4. **Crop วิดีโอ** ไปที่ครึ่งของผู้แสดงท่าทางก่อน pose estimation เพื่อลดสัญญาณรบกวนและปรับปรุงความเชื่อมั่นของ landmark มือ
5. **ลอง chain SEA บน external aligner** ที่ดีกว่า สะท้อน การรวม SAT^{+}+SEA ที่ให้ผล BOBSL สูงสุดของบทความ

บทความอ้างอิง: Z. Jiang, Y. Jang, L. Momeni, G. Varol, S. Ebling, A. Zisserman, *Segment, Embed, and Align: A Universal Recipe for Aligning Subtitles to Signing*, arXiv:2512.08094, 2025.

arxiv.org/abs/2512.08094

เอกสารประกอบใน workspace: example_alignment/README_TH.md, Progress_20042026.md, SEA/README.md, README.md

หนังสืออ้างอิง

[1] H. Bull, T. Afouras, G. Varol, S. Albanie, L. Momeni, A. Zisserman. Aligning subtitles in sign language videos. In *ICCV*, 2021.

[2] Y. Jang, J. Park, S. Choi, J. S. Chung, A. Zisserman. Lost in time: A new temporal benchmark for VideoLLMs. In *ICCV*, 2025.

[3] A. Moryossef, et al. Linguistically motivated sign language segmentation. In *Findings of EMNLP*, 2023.

[4] Z. Jiang, A. Moryossef, M. Müller, S. Ebling. Sign CLIP: Connecting text and sign language by contrastive learning. In *EMNLP*, 2024.

[5] M. Müller, et al. Findings of the WMT 2022 shared task on sign language translation. In *WMT*, 2022.

[6] M. Müller, et al. Findings of the second WMT shared task on sign language translation. In *WMT*, 2023.